

JavaScript Lab — Experiments & Case Studies

Pranav Lakhe | PRN: 24070521248 | Symbiosis Institute of Technology, Nagpur

Experiment 1

Experiment Title

CSS Styling Methods: Internal, Inline, and External CSS

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML Text Editor (VS Code, Sublime Text, Notepad++)

Experiment Program Code

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Types</title>
  <link rel="stylesheet" href="style.css">
  <style>
    body { font-family: Arial, sans-serif; background-color: #f4f4f9; }
    .internal-h1 { color: blue; font-size: 35px; }
    .internal-p { color: red; font-size: 18px; }
    .internal-btn { background-color: #3498db; color: white; padding: 10px 20px; border: none;
border-radius: 5px; }
  </style>
</head>
<body>
  <div style="position:absolute; top:10px; right:20px;">
    <strong>Name:</strong>Pranav Lakhe <br>
    <strong>PRN:</strong> 24070521248
  </div>
  <h2>1. Internal CSS</h2>
  <h1 class="internal-h1">Welcome to Internal CSS</h1>
  <p class="internal-p">This webpage is styled using Internal CSS.</p>
  <button class="internal-btn">Click Here</button>
  <hr>
  <h2>2. Inline CSS</h2>
  <h1 style="color:rgb(46, 250, 5); font-size:35px;">Welcome to Inline CSS</h1>
  <p style="color:rgb(97, 16, 226); font-size:18px;">This webpage is styled using Inline CSS.</p>
  <button style="background:#3498db; color:white; padding:10px 20px; border:none;
border-radius:5px;">Click Here</button>
  <hr>
  <h2>3. External CSS</h2>
  <h1 class="external-h1">Welcome to External CSS</h1>
  <p class="external-p">This webpage is styled using External CSS.</p>
  <button class="external-btn">Click Here</button>
</body>
</html>

/* Style.css */
.external-h1 { color: green; font-size: 35px; }
.external-p { color: purple; font-size: 18px; }
.external-btn { background-color: orange; color: white; padding: 10px 20px; border: none; border-radius:
5px; }
```

Output

The webpage displays three sections demonstrating CSS styling methods — Internal CSS (blue heading, red paragraph), Inline CSS (green heading, purple paragraph), and External CSS (green heading, purple paragraph, orange button) — each rendered with distinct styling to illustrate how the three approaches differ.

Attach the program output/screenshot.

1. Internal CSS

Name: Pranav Lakhe
PRN: 24070521248

Welcome to Internal CSS

This webpage is styled using Internal CSS.

[Click Here](#)

2. Inline CSS

Welcome to Inline CSS

This webpage is styled using Inline CSS.

[Click Here](#)

3. External CSS

Welcome to External CSS

This webpage is styled using External CSS.

[Click Here](#)

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Experiments/Exp1 (24070521248 Pranav Lakhe)/Exp1.html

Result/Conclusion

The three methods of applying CSS — internal, inline, and external — were successfully implemented and verified. External CSS proved most maintainable for reusable styles, inline CSS is useful for one-off overrides, and internal CSS is convenient for page-specific styling without a separate file.

Case Study 1

Case Study Title

Interactive Academic Portal: Linking College and Student Information Systems

Aim

To design and develop a modular, client-side web application using HTML5 and JavaScript that seamlessly links global institutional profiles with dynamic, individual student data validation forms.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository System
- Execution Environment: Any modern web browser (Chrome, Firefox, Edge)

Project Description

This case study implements an interconnected web portal consisting of College.html (institutional landing page) and StudentInfo.html (dynamic student data validation portal). The scripting layer enforces null-check filters on mandatory fields, regex-based PRN format matching, and onsubmit/onclick event handlers.

Case Study Program Code

College.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<title>SIT Nagpur - Institutional Profile</title>
<style>
  body { font-family: Arial, sans-serif; background-color: #f4f7f6; margin: 0; padding: 20px; }
  header { color: #1a5276; padding: 10px 0; border-bottom: 2px solid #e0e0e0; margin-bottom: 20px; }
  h1 { margin: 0; font-size: 32px; }
  p { font-size: 18px; color: #555; }
  .btn { display: inline-block; background: #28a745; color: #fff; padding: 10px 20px; text-decoration:
none; border-radius: 4px; font-weight: bold; border: none; cursor: pointer; }
</style>
</head>
<body>
<header>
  <h1>SIT Nagpur</h1>
  <p>Department of Computer Science and Engineering</p>
</header>
<section id="navigation">
  <a href="StudentInfo.html" class="btn">Student Information</a>
</section>
</body>
</html>

```

StudentInfo.html (validation script)

```

<script>
function validateForm(event) {
  event.preventDefault();
  const prn = document.getElementById("prn").value.trim();
  const fullName = document.getElementById("fullName").value.trim();
  const department = document.getElementById("department").value;

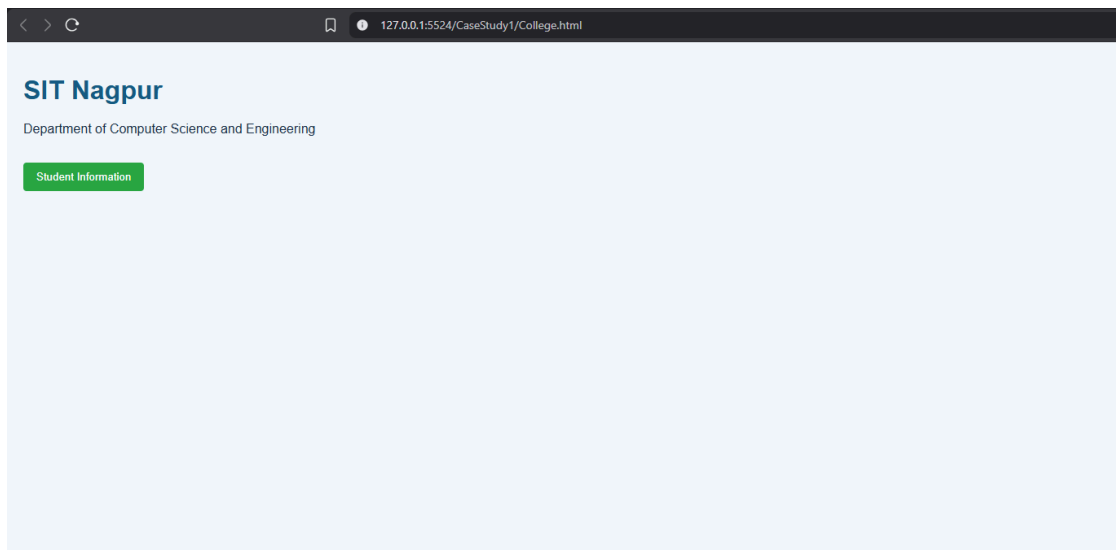
  if (prn === "" || fullName === "" || department === "") {
    alert("Error: All form fields are mandatory and cannot be blank.");
    return false;
  }
  const prnPattern = /^[0-9]+$/;
  if (!prnPattern.test(prn)) {
    alert("Error: PRN must contain numerical values only.");
    return false;
  }
  document.getElementById("outPrn").innerText = prn;
  document.getElementById("outName").innerText = fullName;
  document.getElementById("outDept").innerText = department;
  document.getElementById("outputWindow").style.display = "block";
  return true;
}
</script>

```

Output

The landing interface displays an institutional banner with department listings and a link to the student portal. Invalid submissions trigger an alert dialog; valid submissions render a formatted summary profile (PRN, name, department, verification status) on the page.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy/Case Study 1 (24070521248 Pranav Lakhe)/College.html

Result/Conclusion

The interactive academic portal was successfully implemented, demonstrating page-to-page navigation, client-side form validation with null checks and regex pattern matching, and dynamic DOM updates driven by JavaScript event handlers without any server-side processing.

Experiment 2

Experiment Title

Billing Calculator using ES6 Features (var, let, const, Template Literals, Destructuring)

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML Text Editor (VS Code, Sublime Text, Notepad++)
- JavaScript ES6 Support

Experiment Program Code

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Billing Calculator</title>
  <link rel="stylesheet" href="Style.css">
  <style>
    body { font-family: Arial, sans-serif; background-color: #eef2f7; margin: 0; padding: 40px; }
    .title { color: #003366; }
    .bill-container { background: #fff; padding: 20px; border-radius: 8px; box-shadow: 0 4px 8px
    rgba(0,0,0,0.1); max-width: 400px; margin: 20px auto; }
    .bill-container p { margin: 5px 0; color: #333; }
  </style>
</head>
<body>
  <h2 class="title">Billing Calculator</h2>
  <div class="bill-container" id="billOutput">
    <!-- Bill output will be dynamically injected here -->
  </div>
  <script>
    const GST = 0.18;
    let customerName = prompt("Enter your name:");
    let productName = prompt("Enter the name of product:");
    let quantity = Number(prompt("Enter quantity: "));
    let price = Number(prompt("Enter Price per unit:"));

    let total = quantity * price;
    let getAmount = total * GST;
    let finalAmount = getAmount + total;

    const bill = { customer: customerName, product: productName, quantity, price, total, getAmount,
    finalbill: finalAmount };

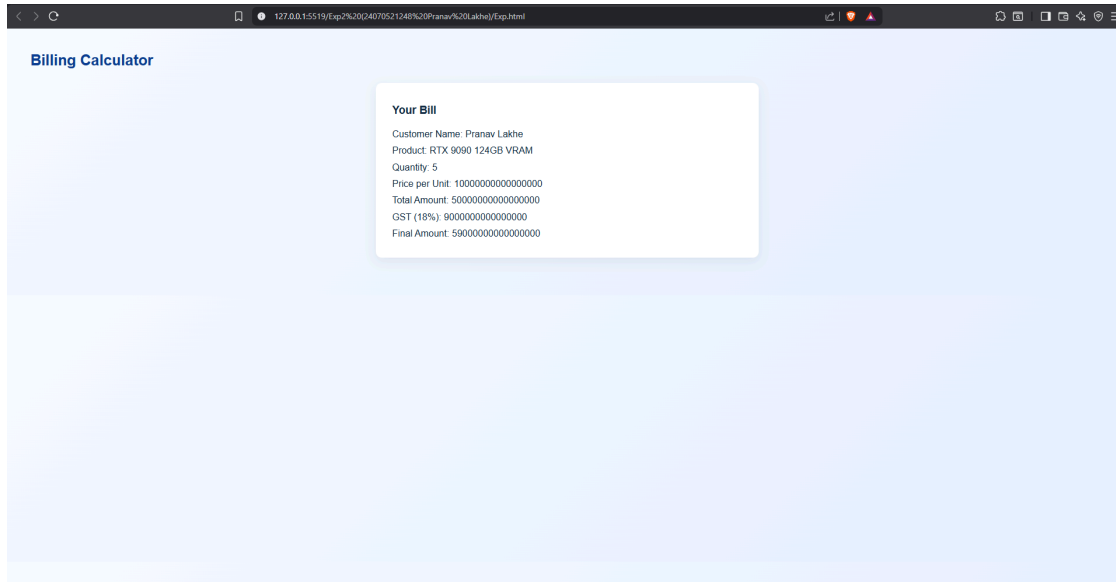
    // Destructuring
    const { customer, product, getAmount: gstAmount = total * GST, finalbill } = bill;

    const outputDiv = document.getElementById('billOutput');
    outputDiv.innerHTML = `
      <h3>Your Bill</h3>
      <p>Customer Name: ${customer}</p>
      <p>Product: ${product}</p>
      <p>Quantity: ${quantity}</p>
      <p>Price per Unit: ${price}</p>
      <p>Total Amount: ${total}</p>
      <p>GST (18%): ${gstAmount}</p>
      <p>Final Amount: ${finalbill}</p>
    `;
  </script>
</body>
</html>
```

Output

The calculator prompts for customer name, product name, quantity, and price per unit, then computes the total, 18% GST, and final amount, displaying a formatted bill using template literals.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Experiments/Exp2 (24070521248 Pranav Lakhe)/Exp.html

Result/Conclusion

The program successfully demonstrates ES6 variable declarations (var, let, const), object destructuring with default values, and template literals for dynamic string interpolation. The bill was generated correctly using the browser prompt-based inputs.

Case Study 2

Case Study Title

Automated E-Commerce Billing Engine & Dynamic Invoice Generator

Aim

To design and develop an interactive, client-side billing application (BILL.HTML) using HTML5, CSS3, and JavaScript (ES6+) that parses real-time product quantities, applies discount thresholds, calculates taxation rates, and renders an itemised invoice directly onto the DOM.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository Hub
- Execution Environment: Any modern web browser

Project Description

This case study builds a transactional checkout application centered around BILL.HTML. It captures item pricing, quantities, and customer details, and processes them entirely client-side — calculating subtotals, applying discount and GST rules, and generating a printable summary instantly.

Case Study Program Code

```
function generateInvoice(event) {
  event.preventDefault();
  const price = parseFloat(document.getElementById("unitPrice").value);
  const qty = parseInt(document.getElementById("quantity").value);
  const discRate = parseFloat(document.getElementById("discountRate").value) || 0;
  const category = document.getElementById("itemCategory").value;

  const rawSubtotal = price * qty;
  const discountAmount = (discRate / 100) * rawSubtotal;
  const discountedSubtotal = rawSubtotal - discountAmount;

  let taxPercentage = 0;
  if (category === "Electronics") taxPercentage = 18;
  else if (category === "Clothing") taxPercentage = 5;
  else if (category === "Essentials") taxPercentage = 0;

  const taxAmount = (taxPercentage / 100) * discountedSubtotal;
  const grandTotal = discountedSubtotal + taxAmount;

  document.getElementById("invSub").innerText = rawSubtotal.toFixed(2);
  document.getElementById("invDisc").innerText = discountAmount.toFixed(2);
  document.getElementById("invTax").innerText = taxAmount.toFixed(2);
  document.getElementById("invGrand").innerText = grandTotal.toFixed(2);
  document.getElementById("invoiceBox").style.display = "block";
}
```

Output

On submission, the application computes the raw subtotal, applies the entered discount percentage, calculates GST based on the selected category (Electronics 18%, Clothing 5%, Essentials 0%), and displays a formatted invoice with subtotal, discount, tax, and grand total.

Attach the program output/screenshot.

Online Shopping Invoice

Customer Information

Customer Name: Pranav Lakhe, Mobile Number: 9604755909, Invoice Number: 24070521248

Product Information

Product Name: rice, Quantity (in Kg): 40, Rate per Kg (₹): 50

Charges & Discounts

Discount (%): 7, GST (%): 18, Packing Charges (₹): 0

Payment & Membership

Payment Mode: ☐ Cash, ☐ Card, ☒ UPI, ☒ Membership (Yes/No)

Bill Summary

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy/Case Study 2 (24070521248 Pranav Lakhe)/BILL.HTML

Result/Conclusion

The billing engine was successfully implemented, demonstrating real-time arithmetic computation, conditional tax-rate assignment, and dynamic invoice rendering entirely in client-side JavaScript.

Experiment 3

Experiment Title

Student Grading System with Dynamic Form Processing

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge, Safari)
- HTML5 with CSS3 support
- JavaScript ES6+ support
- Google Fonts (Inter, JetBrains Mono)

Experiment Program Code

Gradebook.html implements a two-column layout: a form panel for student name, PRN, and subject-wise marks (with dynamic add/remove rows), and a transcript panel. Key logic:

- Input validation for name, 11-digit PRN, and marks (0–100) using regex
- Grade calculation via an if/else-if control structure (Fail, D, C, B, A, A+)
- Remark generation via a switch statement mapped to each grade
- Aggregate total, percentage, and overall grade computed with a for loop
- Dynamic transcript rendering into the DOM with color-coded grade badges

```
function getGradeInfo(marks){
  var letter;
  if(marks <= 50) letter = "Fail";
  else if(marks <= 59) letter = "D";
  else if(marks <= 69) letter = "C";
  else if(marks <= 79) letter = "B";
  else if(marks <= 89) letter = "A";
  else letter = "A+";

  var remark;
  switch(letter){
    case "Fail": remark = "Below passing threshold."; break;
    case "D":    remark = "Passed, needs improvement."; break;
    case "C":    remark = "Satisfactory."; break;
    case "B":    remark = "Good, consistent performance."; break;
    case "A":    remark = "Very good performance."; break;
    case "A+":   remark = "Outstanding performance."; break;
  }
  return { letter: letter, remark: remark };
}
// Full source: Experiments/Exp3 (24070521248 Pranav Lakhe)/Gradebook.html
```

Output

Submitting the form generates an academic transcript showing student name, PRN, date of issue, subject-wise marks and grades, aggregate total, final percentage, overall grade, and Pass/Fail result with a registrar approval section.

Attach the program output/screenshot.

The screenshot displays a web browser window with the address bar showing '127.0.0.1:5500/Exp3/Gradebook.html'. The main content area is divided into two panels. The left panel, titled 'Symbiosis Institute of Technology Student Grading Portal', contains a form for entering student details. The right panel, titled 'INTERNATIONAL INSTITUTE OF TECHNOLOGY OFFICIAL ACADEMIC TRANSCRIPT', shows the generated transcript for Pranav Lakhe.

Symbiosis Institute of Technology Student Grading Portal

Student Name: Pranav Lakhe

PRN (11-digit number): 24070521248

Subject Marks (0-100):

Engineering Mathematics	87	✕
Data Structures & Algorithms	98	✕
Database Management Systems	67	✕
Operating Systems	77	✕
Computer Networks	87	✕

+ Add Subject

Calculate & Generate Report

INTERNATIONAL INSTITUTE OF TECHNOLOGY OFFICIAL ACADEMIC TRANSCRIPT

Student Name: Pranav Lakhe Date of Issue: 22 September 2026

PRN: 24070521248

SUBJECT	MARKS	GRADE
Engineering Mathematics	87 / 100	A
Data Structures & Algorithms	98 / 100	A+
Database Management Systems	67 / 100	C
Operating Systems	77 / 100	B
Computer Networks	87 / 100	A

Aggregate Total: 416 / 500

Final Percentage: 83.20%

Academic Grade: A

Result Status: Passed

Approved REGISTRAR (EVALUATION)

Grading scale : 0-50 Fail · 51-59 D · 60-69 C · 70-79 B · 80-89 A · 90-100 A+

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Exp3/Gradebook.html

Result/Conclusion

The Student Grading System was successfully implemented using vanilla JavaScript, demonstrating conditional control structures (if/else-if and switch), real-time form validation, loop-based computation, and dynamic DOM manipulation to produce a professional transcript.

Case Study 3

Case Study Title

Automated Academic Scholarship Evaluation & Eligibility Matrix Engine

Aim

To design and implement a client-side decision-logic web application (ScholarshipEligibility.html) using HTML5, CSS3, and JavaScript conditional structures that parses academic scores, annual family income thresholds, and demographic parameters to determine scholarship eligibility instantly.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository Hub
- Execution Environment: Any modern web browser

Project Description

This case study implements a rule-based engine evaluating three scholarship schemes (Merit, EBC/Need-based, Special Talent) using nested conditional trapping, compound logical conditions (&&), and dynamic status-class injection to distinguish approved and rejected outcomes.

Case Study Program Code

```
function evaluateEligibility(event) {
    event.preventDefault();
    const score = parseFloat(document.getElementById("academicScore").value);
    let scholarshipType = "";

    if (score >= 90) {
        scholarshipType = "Gold Scholarship";
    } else if (score >= 75) {
        scholarshipType = "Silver Scholarship";
    } else if (score >= 60) {
        scholarshipType = "Bronze Scholarship";
    } else {
        scholarshipType = "No Scholarship";
    }

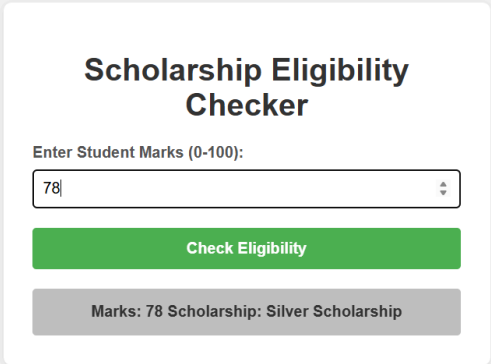
    const panel = document.getElementById("resultPanel");
    panel.style.display = "block";
    panel.className = "result-panel";
    panel.innerText = `Marks: ${score} Scholarship: ${scholarshipType}`;
}
```

Output

On submission, the engine checks the selected scheme's score and income thresholds and displays a result panel: green/approved with a congratulatory message if eligible, or red/rejected with the unmet criteria if not.

Attach the program output/screenshot.

127.0.0.1:5500/caseStudy3.html



Scholarship Eligibility Checker

Enter Student Marks (0-100):

78

Check Eligibility

Marks: 78 Scholarship: Silver Scholarship

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy/Case Study 3 (24070521248 Pranav Lakhe)/ScholarshipEligibility.html

Result/Conclusion

The scholarship eligibility engine was successfully implemented, demonstrating multi-branch conditional logic, compound boolean conditions, and dynamic UI state changes based on evaluation outcomes.

Experiment 4

Experiment Title

Palindrome Checker Suite — Functions, Scope, Closures & Exception Handling

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (arrow functions, const/let)
- Google Fonts (Playfair Display, Plus Jakarta Sans, JetBrains Mono)

Experiment Program Code

```
const PalindromeChecker = (function() {
  let checkCounter = 0;
  let lastExecution = null;

  function reverseString(str) { return str.split('').reverse().join(''); }

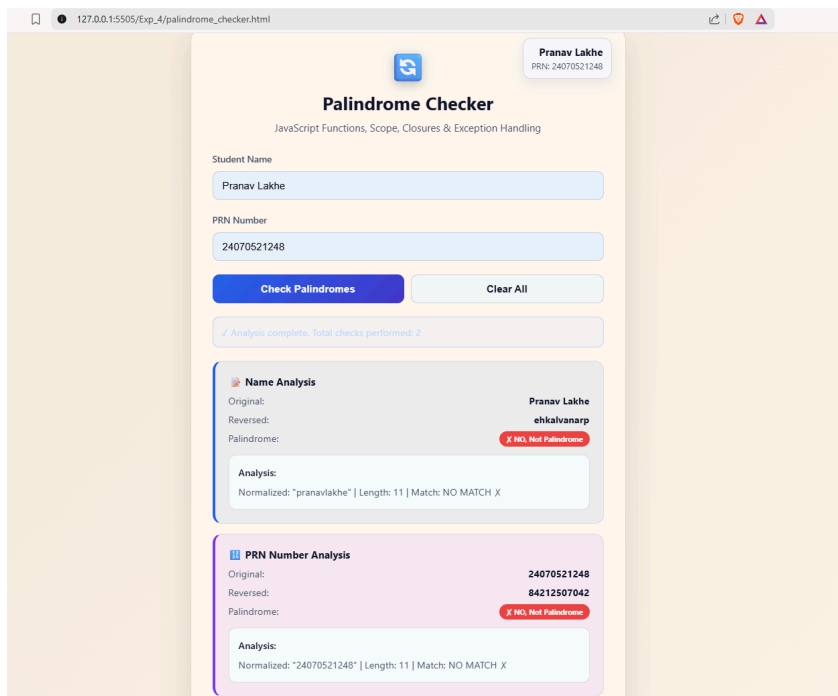
  function normalizeString(str, ignorePunctuation = true, caseSensitive = false) {
    let result = str;
    if (ignorePunctuation) result = result.replace(/[^a-zA-Z0-9]/g, '');
    if (!caseSensitive) result = result.toLowerCase();
    return result;
  }

  function evaluateInput(input, options = {}) {
    if (typeof input !== 'string' || input.trim() === '') {
      throw new Error('Input must be a non-empty text string or sequence.');
```

Output

For each text or number sequence entered, the tool displays the original input, processed (normalized) form, reversed form, character length, and Palindrome/Not-Palindrome status, along with a character-alignment bubble visualization. Example: "Racecar" → Palindrome; "Hello World" → Not Palindrome.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Exp_4/palindrome_checker.html

Result/Conclusion

The Palindrome Checker Suite successfully demonstrates closures via the IIFE module pattern (private counter and timestamp state), robust exception handling with try-catch blocks, and function scoping through helper functions. The implementation validated both text and numeric palindromes correctly, including punctuation- and case-insensitive checks.

Case Study 4

Case Study Title

SecureBank ATM System — PIN Symmetry & Palindrome Verification Module

Aim

To design and implement a client-side ATM PIN verification module (CaseStudy4/index.html) that checks whether an entered PIN is palindromic, using JavaScript function declarations, function expressions, arrow functions, and closures to manage session state.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+, Closures)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository Hub
- Execution Environment: Any modern web browser

Project Description

This case study implements an ATM-style PIN entry terminal with a numeric keypad and sample PIN presets. On verification, the module reverses the entered PIN, compares it digit-by-digit against the original using a position-matching

visualizer, and reports whether the PIN is palindromic (symmetric) or standard (asymmetric). A closure-based session tracker (ATM Session Audit Log) records each attempt, its reversed PIN, symmetry verdict, and timestamp.

Case Study Program Code

```
/**
 * Case Study 4: ATM Card PIN Verification Using JavaScript Functions
 * Demonstrates: Function Declaration, Function Expression, Arrow Function,
 * Scope (global/function/block), and Closures.
 */

// Global scope
const BANK_SYSTEM_NAME = "SecureBank ATM Gateway v4.2";
let globalTotalVerificationsCount = 0;

// Closure: private session state (attempts, palindrome/non-palindrome counts, history)
function createATMSessionTracker() {
  let totalAttempts = 0, palindromeCount = 0, nonPalindromeCount = 0;
  const historyLog = [];

  return {
    recordVerification: function(pin, reversedPin, isPalindrome, securityNotice) {
      totalAttempts++;
      isPalindrome ? palindromeCount++ : nonPalindromeCount++;
      const record = { id: totalAttempts, pin, reversedPin, isPalindrome,
        message: securityNotice.message, securityCode: securityNotice.securityCode,
        timestamp: new Date().toLocaleTimeString() };
      historyLog.unshift(record);
      return record;
    },
    getStats: () => ({ totalAttempts, palindromeCount, nonPalindromeCount }),
    getHistory: () => historyLog,
    resetSession: function() { totalAttempts = 0; palindromeCount = 0; nonPalindromeCount = 0;
    historyLog.length = 0; }
  };
}

const atmSession = createATMSessionTracker();

// Function Declaration
function reversePIN(pin) {
  let reversedStr = "";
  for (let i = pin.length - 1; i >= 0; i--) reversedStr += pin[i]; // block-scoped i
  return reversedStr;
}

// Function Expression
const checkIsPalindrome = function(originalPin, reversedPin) {
  if (!originalPin || originalPin.length === 0) return false;
  return originalPin === reversedPin;
};

// Arrow Function
const generateSecurityAlert = (pin, isPalindrome) => {
  if (isPalindrome) {
    const alertCode = "PAL-SEC-" + Math.floor(1000 + Math.random() * 9000);
    return { title: "SPECIAL BANK SECURITY ALERT",
      message: `PALINDROMIC PIN DETECTED ('${pin}'). This PIN reads identically forward and backward.`,
      statusClass: "status-palindrome", badgeText: "PALINDROMIC SYMMETRY DETECTED",
      securityCode: alertCode };
  }
  const reversed = reversePIN(pin);
  const alertCode = "STD-SEC-" + Math.floor(1000 + Math.random() * 9000);
  return { title: "STANDARD SECURITY CLEARANCE",
    message: `PIN '${pin}' successfully verified. Reversed PIN is '${reversed}'. Standard ATM transaction authorized.`
  };
}
```

```

        statusClass: "status-standard", badgeText: "STANDARD NON-PALINDROME PIN", securityCode:
alertCode });
    };

    // Main verification handler (wired to Verify button)
    function handlePINVerification() {
        const pin = pinInput.value.trim();
        if (!pin) { alert("Please enter or select a PIN number to verify."); return; }

        globalTotalVerificationsCount++;
        const reversedPin = reversePIN(pin);
        const isPal = checkIsPalindrome(pin, reversedPin);
        const notice = generateSecurityAlert(pin, isPal);
        atmSession.recordVerification(pin, reversedPin, isPal, notice);

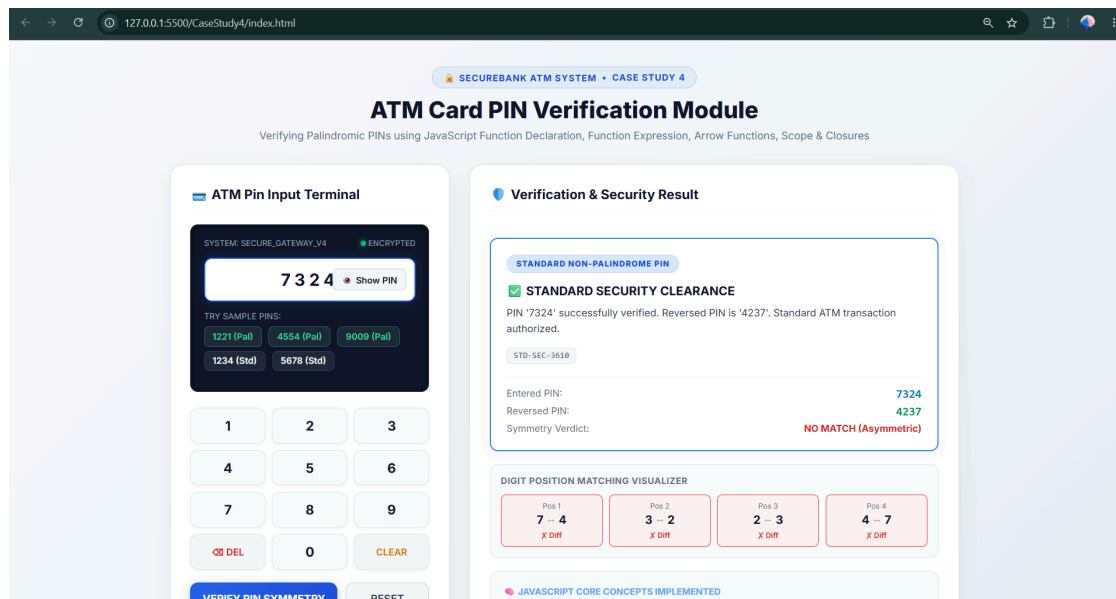
        renderResult(pin, reversedPin, isPal, notice);
        renderDigitVisualizer(pin, reversedPin);
        updateClosureStatsUI();
        renderHistoryUI();
    }
    // Full source: CaseStudy4/index.html & script.js

```

Output

Entering a PIN and clicking "Verify PIN Symmetry" reverses the digits, shows a position-by-position match/diff visualizer, and displays a security clearance banner: green "Standard Security Clearance" for non-palindromic PINs or a distinct palindrome clearance for symmetric ones. Each verification is logged to a running session audit table with attempt number, entered PIN, reversed PIN, symmetry status, and timestamp.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy4/index.html

Result/Conclusion

The ATM PIN verification module was successfully implemented, demonstrating function declarations, function expressions, and arrow functions working together, along with closures for maintaining session-scoped attempt counters and audit history across multiple verifications.

Experiment 5

Experiment Title

Shopping Cart Calculator & Receipt Generator (Arrays & Objects)

Software/Tools Required

- Node.js (for console version)
- Web Browser (Chrome, Firefox, Edge)
- HTML5, CSS3, JavaScript ES6+
- Google Fonts (Plus Jakarta Sans, JetBrains Mono)

Experiment Program Code

```
const readline = require('readline');
const rl = readline.createInterface({ input: process.stdin, output: process.stdout });

let cart = [];
let appliedCoupon = null;

const COUPONS = {
  'SAVE10': { type: 'percent', value: 10 },
  'SUPER20': { type: 'percent', value: 20 },
  'FLAT50': { type: 'fixed', value: 50 },
  'WELCOME10': { type: 'fixed', value: 10 }
};

function askItem() {
  rl.question('Enter product name (or type "done" to finish): ', (name) => {
    if (name.toLowerCase() === 'done') { askCoupon(); return; }
    rl.question('Enter product price: ', (priceInput) => {
      const price = parseFloat(priceInput);
      if (isNaN(price)) { console.log("Invalid price."); return askItem(); }
      rl.question('Enter product quantity: ', (qtyInput) => {
        const quantity = parseInt(qtyInput, 10);
        if (isNaN(quantity)) { console.log("Invalid quantity."); return askItem(); }
        cart.push({ name, price, quantity });
        console.log(`Added ${quantity}x '${name}' to the cart.\n`);
        askItem();
      });
    });
  });
}

function askCoupon() {
  rl.question('Enter coupon code (SAVE10, SUPER20, FLAT50) or press Enter to skip: ', (code) => {
    const cleanCode = code.trim().toUpperCase();
    if (cleanCode && COUPONS[cleanCode]) {
      appliedCoupon = { code: cleanCode, ...COUPONS[cleanCode] };
      console.log(`✓ Coupon '${cleanCode}' applied!\n`);
    } else if (cleanCode) {
      console.log("Invalid coupon code. Continuing without coupon...\n");
    }
    calculateAndPrint();
    rl.close();
  });
}

function calculateAndPrint() {
  if (cart.length === 0) { console.log("\nYour cart is empty!"); return; }
  console.log("\n--- Receipt ---");
  cart.forEach(item => console.log(`${item.name} - ${item.price} x ${item.quantity}`));
  console.log("-----");

  let total = cart.reduce((sum, item) => sum + (item.price * item.quantity), 0);
  let bulkDiscount = total > 1000 ? total * 0.10 : 0;
```

```

    let couponDiscount = 0;
    if (appliedCoupon) {
      if (appliedCoupon.type === 'percent') couponDiscount = (total - bulkDiscount) *
(appliedCoupon.value / 100);
      else if (appliedCoupon.type === 'fixed') couponDiscount = Math.min(appliedCoupon.value, total -
bulkDiscount);
    }

    let finalAmount = total - bulkDiscount - couponDiscount;
    console.log(`Total Amount: ${total}`);
    if (bulkDiscount > 0) console.log(`Bulk Discount (10%): ${bulkDiscount}`);
    if (appliedCoupon) console.log(`Coupon Discount (${appliedCoupon.code}): ${couponDiscount}`);
    console.log(`Final Amount: ${finalAmount}`);
  }

  console.log("Pranav Lakhe | PRN: 24070521248");
  console.log("--- Shopping Cart Calculator ---");
  askItem();

  // Browser UI version (index.html + styles.css) mirrors this logic with a
  // form-based cart, promo-code pills (SAVE10/SUPER20/FLAT50/WELCOME10),
  // a bulk-discount progress bar, and a generated receipt modal.

```

Output

The console version prompts for product name, price, and quantity in a loop, then applies a 10% bulk discount (orders over \$1000) and any entered coupon code, printing an itemized receipt with total, discounts, and final amount. The browser version mirrors this with a live cart table, discount progress bar, and a printable receipt modal.

Attach the program output/screenshot.

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Exp5/index.html (UI) & cart.js (console logic)

Result/Conclusion

The Shopping Cart Calculator successfully demonstrates array methods (push, reduce) and object-based data modeling for cart items and coupons, along with conditional discount logic — confirming correct order-total, discount, and final-amount computation across both the console and browser interfaces.

Case Study 5

Case Study Title

Arrays & Loops in JavaScript — Manipulation, Higher-Order Methods, and a Max/Min Finder

Aim

To explore core JavaScript array manipulation methods (push, pop, unshift, shift, splice, slice) and higher-order array methods (forEach, map, filter, reduce) through an interactive visual demo, culminating in a case study that finds the maximum and minimum values in an array.

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Array methods)

Project Description

Section A visually demonstrates mutating array methods (push/pop/unshift/shift/splice) and the non-mutating slice() with live before/after console-style logs and highlighted array-item chips. Section B demonstrates the higher-order methods forEach, map, filter, and reduce on a fixed sample array. The embedded case study applies reduce() twice to find the maximum and minimum of an array, highlighting them in green and red respectively.

Case Study Program Code

```
// Section A: Array Manipulation
const INITIAL_ARRAY = [25, 10, 45, 5, 30, 15];
let arr = [...INITIAL_ARRAY];

document.getElementById('btn-push').addEventListener('click', () => {
  const val = Math.floor(Math.random() * 50) + 1;
  arr.push(val);
  renderArray(arrayVisual, arr, [arr.length - 1], 'highlight-new');
});

document.getElementById('btn-splice').addEventListener('click', () => {
  const startIdx = 1, deleteCount = 1, insertVal = Math.floor(Math.random() * 50) + 1;
  const spliced = arr.splice(startIdx, deleteCount, insertVal);
  renderArray(arrayVisual, arr, [startIdx], 'highlight-new');
});

// Section B: Higher-Order Methods
const methodArr = [25, 10, 45, 5, 30, 15];

document.getElementById('btn-map').addEventListener('click', () => {
  const doubled = methodArr.map(val => val * 2);
  out.textContent = `Original: [${methodArr.join(', ')}]\nDoubled: [${doubled.join(', ')}]`;
});

document.getElementById('btn-filter').addEventListener('click', () => {
  const filtered = methodArr.filter(val => val > 15);
  out.textContent = `Result: [${filtered.join(', ')}]`;
});

document.getElementById('btn-reduce').addEventListener('click', () => {
  const sum = methodArr.reduce((acc, val) => acc + val, 0);
  out.textContent = `Sum = ${sum}`;
});

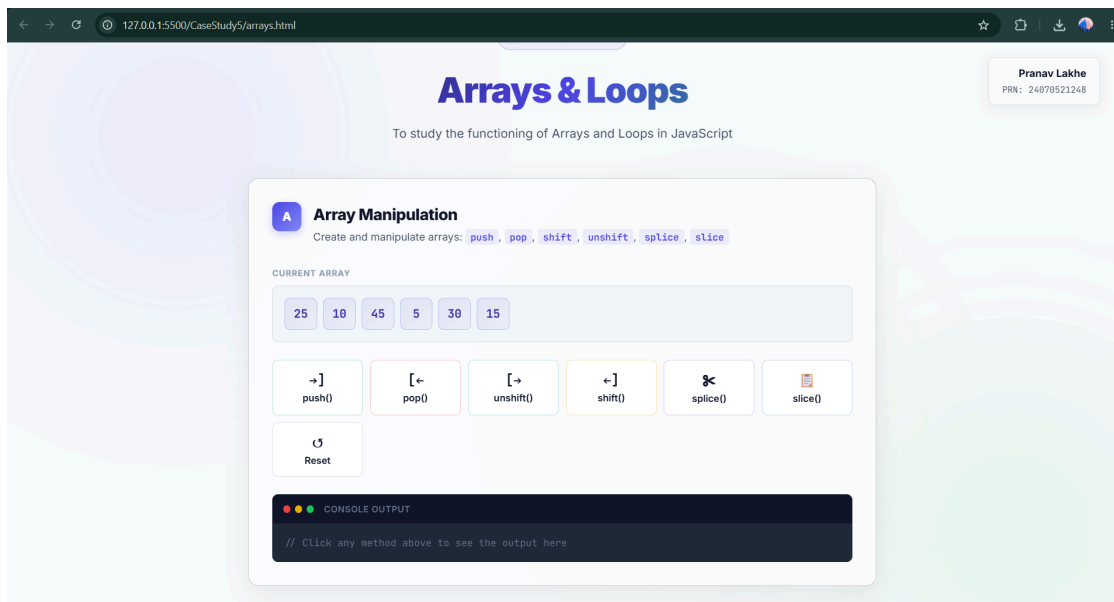
// Case Study: Max & Min Finder
document.getElementById('btn-run-case').addEventListener('click', () => {
  const numbers = [25, 10, 45, 5, 30, 15];
  const maxVal = numbers.reduce((max, val) => val > max ? val : max, numbers[0]);
```

```
const minVal = numbers.reduce((min, val) => val < min ? val : min, numbers[0]);
csMaxEl.textContent = maxVal;
csMinEl.textContent = minVal;
});
```

Output

Each button click logs the operation performed, the array before and after, and updates the visual array chips with highlights. The Max/Min case study displays result cards showing the computed maximum (25... 45 depending on data) and minimum values, with the corresponding array elements highlighted.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy5/arrays.html

Result/Conclusion

The case study successfully demonstrates both direct array mutation methods and functional, non-mutating higher-order methods, and confirms `reduce()` as an effective accumulator pattern for finding extremal values in a dataset.

Experiment 6

Experiment Title

String Functions & Regex Studio — Text Analytics and String Transformation Suite

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (String methods, Regex)

Experiment Program Code

Reconstructed from the recorded output below — the app has two tabs (String Manipulator and Regex Studio); the String Manipulator tab computes live text statistics and applies transform operations:

```
// Live statistics, recalculated on every input event
function updateStats() {
  const text = inputBox.value;
  charCount.textContent = text.length;
  wordCount.textContent = (text.trim().match(/\S+/g) || []).length;
  sentenceCount.textContent = (text.match(/[.!?]+/g) || []).length || (text.trim() ? 1 : 0);
  lineCount.textContent = text.split('\n').length;
  spaceCount.textContent = (text.match(/ /g) || []).length;
  const words = wordCount.textContent;
  readTime.textContent = Math.max(1, Math.round(words / 200)) + ' m';
}
inputBox.addEventListener('input', updateStats);

// Transform Operations
const transforms = {
  UPPERCASE: (s) => s.toUpperCase(),
  lowercase: (s) => s.toLowerCase(),
  'Title Case': (s) => s.replace(/\w\S*/g, w => w[0].toUpperCase() + w.slice(1).toLowerCase()),
  Reverse: (s) => s.split('').reverse().join(''),
  camelCase: (s) => s.toLowerCase().replace(/^[^a-zA-Z0-9]+(.)/g, (_, c) => c.toUpperCase()),
  'kebab-case': (s) => s.trim().toLowerCase().replace(/^[^a-zA-Z0-9]+/g, '-'),
  snake_case: (s) => s.trim().toLowerCase().replace(/^[^a-zA-Z0-9]+/g, '_'),
  'Trim Whitespace': (s) => s.trim().replace(/\s+/g, ' ')
};

document.querySelectorAll('.transform-btn').forEach(btn => {
  btn.addEventListener('click', () => {
    const operation = btn.dataset.operation;
    outputBox.textContent = transforms[operation](inputBox.value);
  });
});

copyOutputBtn.addEventListener('click', () => {
  navigator.clipboard.writeText(outputBox.textContent);
});

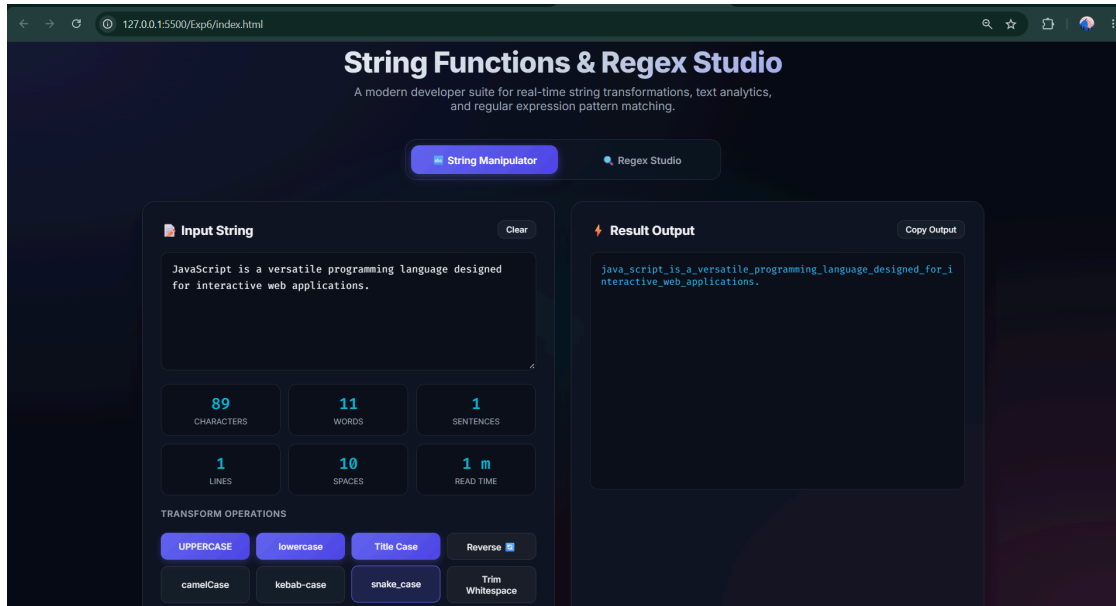
// Regex Studio tab: pattern tester, email validator, extractor
regexTestBtn.addEventListener('click', () => {
  const pattern = new RegExp(regexInput.value, regexFlags.value);
  const matches = Array.from(sampleText.value.matchAll(pattern), m => m[0]);
  regexResult.textContent = matches.length ? matches.join(', ') : 'No matches found';
});
```

Output

Typing or pasting text into the Input String panel live-updates six stat tiles (Characters, Words, Sentences, Lines, Spaces, Read Time). Clicking a Transform Operation button (UPPERCASE, lowercase, Title Case, Reverse, camelCase, kebab-case, snake_case, Trim Whitespace) applies that transformation and shows the result in the Result Output panel, e.g. snake_case on the sample sentence produced

"java_script_is_a_versatile_programming_language_designed_for_interactive_web_applications."

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Exp6/index.html

Result/Conclusion

This experiment successfully demonstrates live text analytics (character/word/sentence/line counts, reading-time estimation) driven by input events, alongside a suite of string transformation utilities and a regex pattern tester — confirming JavaScript's string and RegExp APIs as effective tools for building interactive text-processing utilities.

Case Study 6

Case Study Title

String Operations & Event-Driven Form Validation

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (String methods, DOM Events)

Project Description

This case study is split into two parts: (1) String Operations — reversing a paragraph of text, counting vowels, and replacing all occurrences of a word using regex; and (2) Event Handling — a registration form (name, email, phone) validated live on focus, blur, and change events, plus a full validation pass on submit.

Case Study Program Code

```
// 1. String Operations
document.getElementById('reverseBtn').addEventListener('click', () => {
  const text = textInput.value.trim();
  const reversed = text.split('').reverse().join('');
  stringOutput.innerHTML = '<strong>Reversed String:</strong> ' + reversed;
});

document.getElementById('vowelBtn').addEventListener('click', () => {
```

```

const text = textInput.value.toLowerCase();
const vowels = text.match(/[aeiou]/g) || [];
stringOutput.innerHTML = '<strong>Vowels Found:</strong> ' + vowels.length;
});

document.getElementById('replaceBtn').addEventListener('click', () => {
  const updatedText = textInput.value.replace(/JavaScript/gi, 'JS');
  stringOutput.innerHTML = '<strong>Updated Text:</strong><br>' + updatedText;
});

// 2. Event Handling & Validation
const validateEmail = () => {
  const value = fieldMap.email.value.trim();
  const pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return pattern.test(value);
};

const validatePhone = () => {
  const pattern = /^\d{10}$/;
  return pattern.test(fieldMap.phone.value.trim());
};

Object.entries(fieldMap).forEach(([key, field]) => {
  field.addEventListener('focus', () => field.closest('.field').classList.add('active'));
  field.addEventListener('blur', () => { /* run matching validator */ });
  field.addEventListener('change', () => { /* run matching validator */ });
});

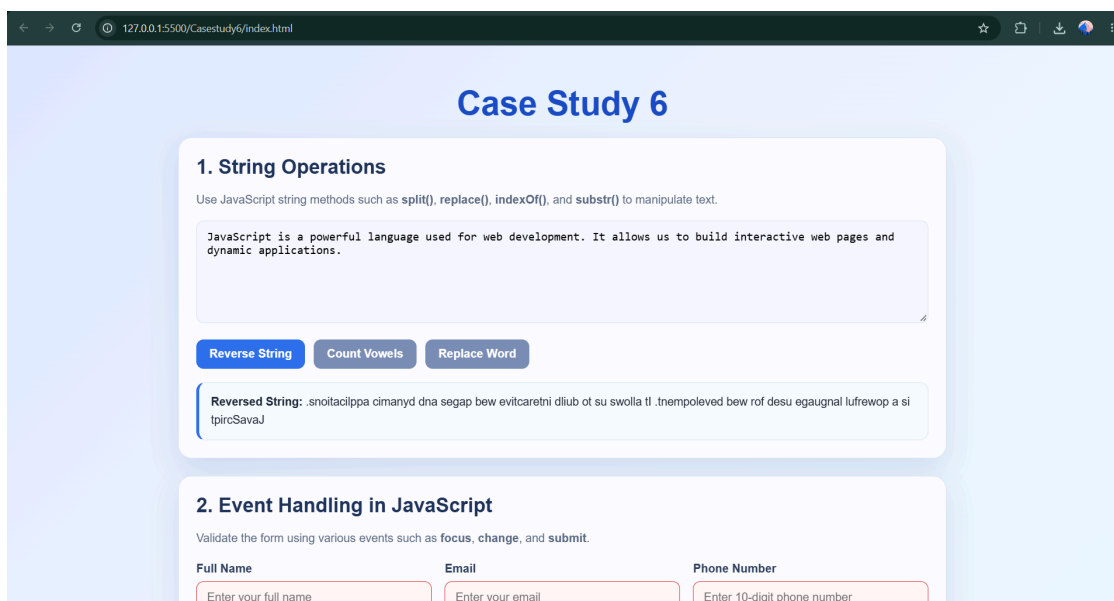
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const isValid = validateName() && validateEmail() && validatePhone();
  formStatus.textContent = isValid ? 'Form submitted successfully!' : 'Please correct the highlighted fields.';
});
// Full source: CaseStudy/Case Study 6 (24070521248 Pranav Lakhe)/index.html

```

Output

The String Operations panel displays the reversed text, vowel count and list, or the text with "JavaScript" replaced by "JS" depending on the button clicked. The registration form highlights each field green (valid) or red (invalid) as the user interacts with it, and shows a success or error banner on submit.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Casestudy6/index.html

Result/Conclusion

This case study successfully demonstrates core string manipulation methods (split, reverse, join, match, replace) alongside a full event-driven validation lifecycle (focus, blur, change, submit), confirming JavaScript's ability to deliver responsive, field-level form feedback.

Experiment 7

Experiment Title

Task List Manager using DOM Traversal and Dynamic Element Creation

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (DOM APIs)

Experiment Program Code

```
const form = document.querySelector("#task-form");
const input = document.querySelector("#task-input");
const taskSections = document.querySelectorAll(".task-section");
const taskCount = document.querySelector("#task-count");

function escapeHtml(unsafeText) {
    return unsafeText
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#039;");
}

function updateSummary() {
    const tasks = document.querySelectorAll(".task");
    taskCount.textContent = `${tasks.length} ${tasks.length === 1 ? "task" : "tasks"}`;
    taskSections.forEach((section) => {
        section.querySelector(".section-empty").hidden = section.querySelectorAll(".task").length > 0;
    });
}

function createTask(text) {
    const task = document.createElement("li");
    task.className = "task";
    const safeText = escapeHtml(text);
    task.innerHTML = `
        <span class="task-check" aria-hidden="true"></span>
        <span class="task-text">${safeText}</span>
        <span class="task-actions">
            <select class="status-select"><option value="pending">Pending</option>
            <option value="in-progress">In progress</option>
            <option value="completed">Completed</option></select>
            <button class="edit" type="button">Edit</button>
            <button class="delete" type="button">Delete</button>
        </span>`;
    document.querySelector("#pending-list").append(task);
    updateSummary();
}

form.addEventListener("submit", (event) => {
    event.preventDefault();
    const text = input.value.trim();
    if (!text) return;
    createTask(text);
    input.value = "";
});

document.querySelector("section[aria-labelledby='list-title']").addEventListener("click", (event) => {
    const action = event.target.closest("button");
    const task = event.target.closest(".task");
    if (!action || !task) return;
```

```

    if (action.classList.contains("delete")) task.remove();
    else if (action.classList.contains("edit")) {
        const taskText = task.querySelector(".task-text");
        const revisedText = window.prompt("Edit task", taskText.textContent);
        if (revisedText !== null && revisedText.trim()) taskText.textContent =
escapeHtml(revisedText.trim());
    }
    updateSummary();
});

document.querySelector("section[aria-labelledby='list-title']").addEventListener("change", (event) => {
    if (!event.target.classList.contains("status-select")) return;
    const task = event.target.closest(".task");
    const destination = document.querySelector(`#${event.target.value}-list`);
    task.classList.toggle("completed", event.target.value === "completed");
    destination.append(task);
    updateSummary();
});

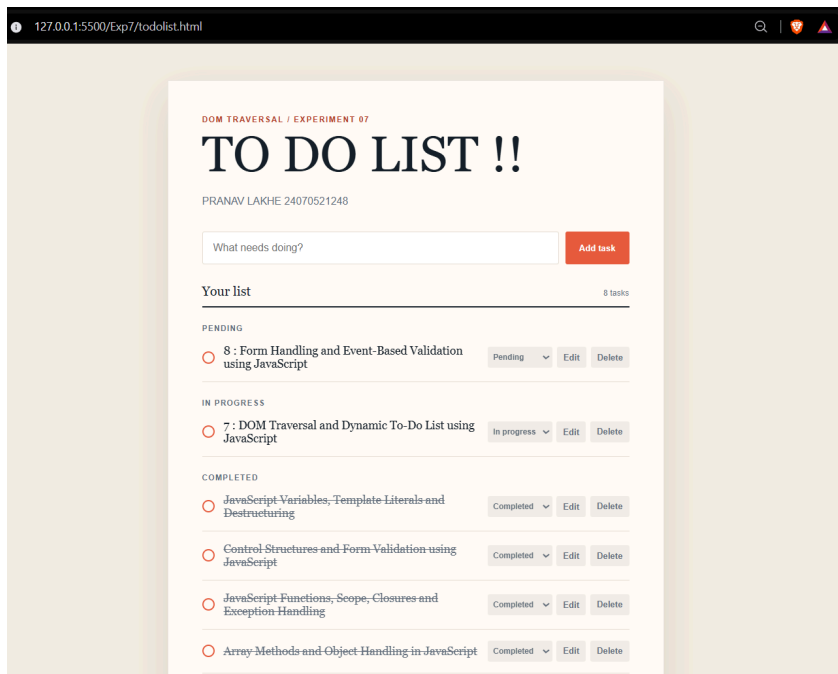
updateSummary();

```

Output

New tasks entered in the input field are appended to a Pending column. Each task row lets the user change its status (Pending/In Progress/Completed) via a dropdown — moving it between the corresponding list sections — edit its text via a prompt dialog, or delete it. A running task count is kept in sync with every change.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Exp7/todolist.html

Result/Conclusion

The DOM traversal exercise was successfully implemented, demonstrating createElement()-based node injection, closest()-based event delegation for dynamically added elements, and querySelector-driven traversal between sibling containers — confirming a solid grasp of DOM manipulation beyond static markup.

Case Study 7

Case Study Title

Form Handling with JavaScript — Multi-Field Signup with Password Strength & Theme Toggle

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Form Events, Regex)

Project Description

This case study implements a full signup form with dynamically populated Day/Month/Year dropdowns, live password-strength scoring, show/hide password toggles, a light/dark theme switch, and toast-notification feedback, all validated on submit.

Case Study Program Code

```
// Populate Day (1-31) and Year (2026-1950) dropdowns dynamically
const daySelect = document.getElementById('day');
for (let i = 1; i <= 31; i++) {
  const opt = document.createElement('option');
  opt.value = i; opt.textContent = i;
  daySelect.appendChild(opt);
}

// Live Password Strength Meter
passwordInput.addEventListener('input', function() {
  const val = this.value;
  let score = 0;
  if (val.length >= 6) score++;
  if (val.length >= 10) score++;
  if (/[A-Z]/.test(val)) score++;
  if (/[0-9]/.test(val)) score++;
  if (/^[A-Za-z0-9]/.test(val)) score++;

  if (score <= 2) { strengthBar.style.width = '33%'; strengthBar.style.backgroundColor = '#ef4444'; }
  else if (score <= 4) { strengthBar.style.width = '66%'; strengthBar.style.backgroundColor = '#f59e0b'; }
  else { strengthBar.style.width = '100%'; strengthBar.style.backgroundColor = '#10b981'; }
});

// Theme Toggle
themeToggleBtn.addEventListener('click', () => {
  const currentTheme = document.documentElement.getAttribute('data-theme');
  const newTheme = currentTheme === 'light' ? 'dark' : 'light';
  document.documentElement.setAttribute('data-theme', newTheme);
});

// Submit validation
document.getElementById('signupForm').addEventListener('submit', function (e) {
  e.preventDefault();
  resetErrors();
  const firstname = document.getElementById('firstname').value.trim();
  // ... validates firstname, DOB, username, email, password, etc.
  // shows toast on success, inline errors on failure
});
// Full source: CaseStudy/Case Study 7 (24070521248 Pranav Lakhe)/LoginForm.html
```

Output

As the user types a password, a colored strength bar updates live (red/orange/green). The eye icon toggles password visibility. Submitting with missing/invalid fields highlights each one with an inline error; a successful submit shows a toast notification. The theme button switches the whole form between light and dark palettes.

Attach the program output/screenshot.

PL Pranav Lakhe
PRN: 24070521248 • Case Study 7

JavaScript Form Handling

Create Account

• Mandatory fields must be completed

Firstname * Lastname

John Doe

Birthday *

Day Month Year

Username * E-mail *

johndoe john@example.com

Website *

https://example.com

Password * Re-password *

☐ I agree to the [terms & conditions](#).

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: casestudy7/LoginForm.html

Result/Conclusion

This case study successfully demonstrates dynamic option population, live input-driven UI feedback (password strength), multi-field submit-time validation, and CSS-variable-based theme switching — combining several JavaScript event-handling patterns in a single realistic signup form.

Experiment 8

Experiment Title

VALENCE Athletic Club — Multi-Field Admission Form with Real-Time Validation

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Form validation, Regex)

Experiment Program Code

```
const PLAN_DETAILS = {
  basic: { name: "Basic", duration: "1 Month", price: "₹1,499" },
  pro: { name: "Pro Athlete", duration: "6 Months", price: "₹6,999" },
  elite: { name: "Elite VIP", duration: "1 Year", price: "₹11,999" }
};

function validateName() {
  const val = nameInput.value.trim();
  if (val.length === 0) { clearState(nameInput, nameError, nameIndicator); return false; }
  if (val.length >= 3) { setValid(nameInput, nameError, nameIndicator); return true; }
  setInvalid(nameInput, nameError, nameIndicator, "Full name must be at least 3 characters.");
  return false;
}

function validateEmail() {
  const val = emailInput.value.trim();
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (val.length === 0) { clearState(emailInput, emailError, emailIndicator); return false; }
  if (emailPattern.test(val)) { setValid(emailInput, emailError, emailIndicator); return true; }
  setInvalid(emailInput, emailError, emailIndicator, "Please enter a valid email address.");
  return false;
}

function validateMobile() {
  mobileInput.value = mobileInput.value.replace(/\D/g, '');
  const val = mobileInput.value;
  if (val.length === 0) { clearState(mobileInput, mobileError, mobileIndicator); return false; }
  if (val.length === 10) { setValid(mobileInput, mobileError, mobileIndicator); return true; }
  setInvalid(mobileInput, mobileError, mobileIndicator, `Mobile number must be 10 digits (${val.length}/10).`);
  return false;
}

function validateAge() {
  const val = ageInput.value;
  if (val === "") { clearState(ageInput, ageError, ageIndicator); return false; }
  const num = parseInt(val, 10);
  if (!isNaN(num) && num >= 16 && num <= 100) { setValid(ageInput, ageError, ageIndicator); return true; }
  setInvalid(ageInput, ageError, ageIndicator, "You must be 16 or older to join.");
  return false;
}

// Full source: Experiments/Exp8 (24070521248) Pranav Lakhe/exp8html
```

Output

As the user types, each field (name, email, mobile, age, plan) is validated live — showing a green check or a red error message and updating a progress bar. On successful submission, a modal "membership pass" appears summarizing the applicant's name, email, mobile, and selected plan tier with a generated ID.

Attach the program output/screenshot.

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Experiments/Exp8 (24070521248) Pranav Lakhe/exp8.html

Result/Conclusion

The admission form successfully demonstrates real-time, field-level validation using regex patterns and numeric range checks, along with dynamic UI feedback (indicators, progress bar, modal). This confirms JavaScript's capability to deliver a responsive, user-friendly validation experience without page reloads.

Case Study 8

Case Study Title

Online Job Application Form — Sectioned Validation with Live Progress Tracking

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Form Validation)

Project Description

This case study implements a four-section job application form — Personal Information, Educational Qualification, Professional Information, and Technical Skills — with a live completion progress bar that updates as fields are filled, a theme toggle, and full field-by-field validation on submit.

Case Study Program Code

```

function updateProgress() {
    const requiredInputs = form.querySelectorAll('input[type="text"], input[type="email"],
input[type="number"], input[type="date"], select, textarea');
    let filledCount = 0;
    requiredInputs.forEach(input => { if (input.value.trim() !== '') filledCount++; });
    if (document.querySelector('input[name="gender"]:checked')) filledCount++;
    const percentage = Math.round((filledCount / (requiredInputs.length + 1)) * 100);
    progressBar.style.width = percentage + '%';
}
form.addEventListener('input', updateProgress);
form.addEventListener('change', updateProgress);

function validateField(groupId, fieldId, condition) {
    const group = document.getElementById(groupId);
    const errorElem = group.querySelector('.error');
    if (!condition) { group.classList.add('invalid'); if (errorElem) errorElem.style.display = 'flex';
isValid = false; }
    else { group.classList.remove('invalid'); if (errorElem) errorElem.style.display = 'none'; }
}

form.addEventListener('submit', function (e) {
    e.preventDefault();
    let isValid = true;
    const email = document.getElementById('email').value.trim();
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    validateField('group-email', 'email', emailRegex.test(email));

    const mobile = document.getElementById('mobile').value.trim();
    validateField('group-mobile', 'mobile', /^[0-9]{10}$/ .test(mobile));

    const passingYear = parseInt(document.getElementById('passingYear').value, 10);
    validateField('group-passingYear', 'passingYear', passingYear >= 1950 && passingYear <= 2026);
    // ... additional field checks across all four sections

    if (isValid) { successMsg.style.display = 'block'; form.reset(); progressBar.style.width = '0%'; }
});
// Full source: CaseStudy/Case Study 8 (24070521248 Pranav Lakhe)/Job_Application_Form.html

```

Output

As fields are filled, the progress bar advances toward 100%. Submitting with any invalid or empty required field highlights that field's group in red with an inline error and blocks submission. When every section passes validation, a success message is shown, the form resets, and the page scrolls to the top.

Attach the program output/screenshot.

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy8/Job_Application_Form.html

Result/Conclusion

This case study successfully demonstrates section-by-section validation of a large multi-part form, a reusable `validateField()` helper pattern, and live progress tracking tied to input/change events — confirming a scalable approach to validating complex forms in vanilla JavaScript.

Experiment 9

Experiment Title

Theme Settings — Persisting User Preference with localStorage and sessionStorage

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Web Storage API)

Experiment Program Code

```
((() => {
  const sessionTheme = sessionStorage.getItem('preferredTheme');
  const localTheme = localStorage.getItem('preferredTheme');
  const theme = sessionTheme || localTheme || 'light';
  document.documentElement.dataset.theme = theme === 'dark' ? 'dark' : 'light';
})();

const themeSelect = document.getElementById('theme-select');
const statusMessage = document.getElementById('status-message');
const validThemes = new Set(['light', 'dark']);

function applyTheme(theme) {
  const selectedTheme = validThemes.has(theme) ? theme : 'light';
  document.documentElement.dataset.theme = selectedTheme;
  themeSelect.value = selectedTheme;
}

document.getElementById('save-local').addEventListener('click', () => {
  const theme = themeSelect.value;
  localStorage.setItem('preferredTheme', theme);
  sessionStorage.removeItem('preferredTheme');
  applyTheme(theme);
  statusMessage.textContent = 'Theme saved using localStorage.';
});

document.getElementById('save-session').addEventListener('click', () => {
  const theme = themeSelect.value;
  sessionStorage.setItem('preferredTheme', theme);
  localStorage.removeItem('preferredTheme');
  applyTheme(theme);
  statusMessage.textContent = 'Theme saved using sessionStorage.';
});

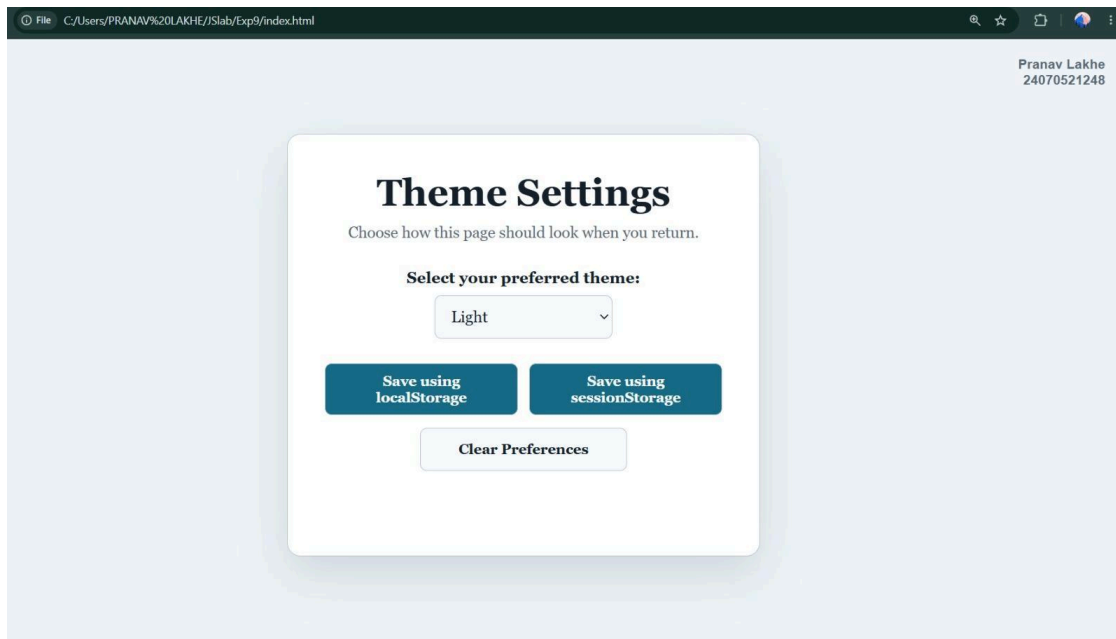
document.getElementById('clear-preferences').addEventListener('click', () => {
  localStorage.removeItem('preferredTheme');
  sessionStorage.removeItem('preferredTheme');
  applyTheme('light');
  statusMessage.textContent = 'Preferences cleared.';
});

const savedTheme = sessionStorage.getItem('preferredTheme') || localStorage.getItem('preferredTheme');
applyTheme(savedTheme || 'light');
if (savedTheme && validThemes.has(savedTheme)) {
  const storageName = sessionStorage.getItem('preferredTheme') ? 'sessionStorage' : 'localStorage';
  statusMessage.textContent = `Theme restored from ${storageName}.`;
}
```

Output

Selecting Light/Dark and clicking "Save using localStorage" persists the choice across browser restarts; "Save using sessionStorage" persists it only for the current tab session. On reload, the page automatically restores the previously saved theme and reports which storage mechanism it came from. "Clear Preferences" resets both stores to Light.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Exp9/index.html

Result/Conclusion

The experiment successfully demonstrates the distinction between localStorage (persists indefinitely) and sessionStorage (cleared when the tab closes) for client-side state persistence, along with IIFE-based early theme application to avoid a flash of unstyled content on load.

Case Study 9

Case Study Title

To-Do List with Theme Toggle — Category Tags, Search, Filtering & localStorage Persistence

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (localStorage, Array methods)

Project Description

This case study builds on the basic to-do list concept by adding category tags (Work/Urgent/Personal/Low), a live search box, All/Active/Completed filtering, a completion progress bar, and persistence of the entire task list to localStorage so it survives page reloads. A light/dark theme toggle is also included.

Case Study Program Code

```
let tasks = JSON.parse(localStorage.getItem('cs9_tasks')) || [
  { id: 1, text: 'Complete JS Lab Experiment 10 submission', category: 'Work', completed: true },
  { id: 2, text: 'Review Case Study 9 Theme Toggle features', category: 'Urgent', completed: false }
];

function saveTasks() { localStorage.setItem('cs9_tasks', JSON.stringify(tasks)); }

function renderTasks() {
  taskList.innerHTML = '';
```

```

    const filteredTasks = tasks.filter(task => {
      const matchesFilter = activeFilter === 'all' ? true : activeFilter === 'active' ?
!task.completed : task.completed;
      const matchesSearch = task.text.toLowerCase().includes(searchQuery.toLowerCase());
      return matchesFilter && matchesSearch;
    });
    filteredTasks.forEach(task => {
      const li = document.createElement('li');
      li.className = `task-item ${task.completed ? 'completed' : ''}`;
      const badgeClass = task.category === 'Urgent' ? 'badge-urgent' : task.category === 'Work' ?
'badge-work' : 'badge-personal';
      li.innerHTML = `<input type="checkbox" ${task.completed ? 'checked' : ''} data-id="${task.id}">
      <span class="task-text">${escapeHtml(task.text)}</span>
      <span class="task-badge ${badgeClass}">${task.category}</span>`;
      taskList.appendChild(li);
    });
    updateMetrics();
  }

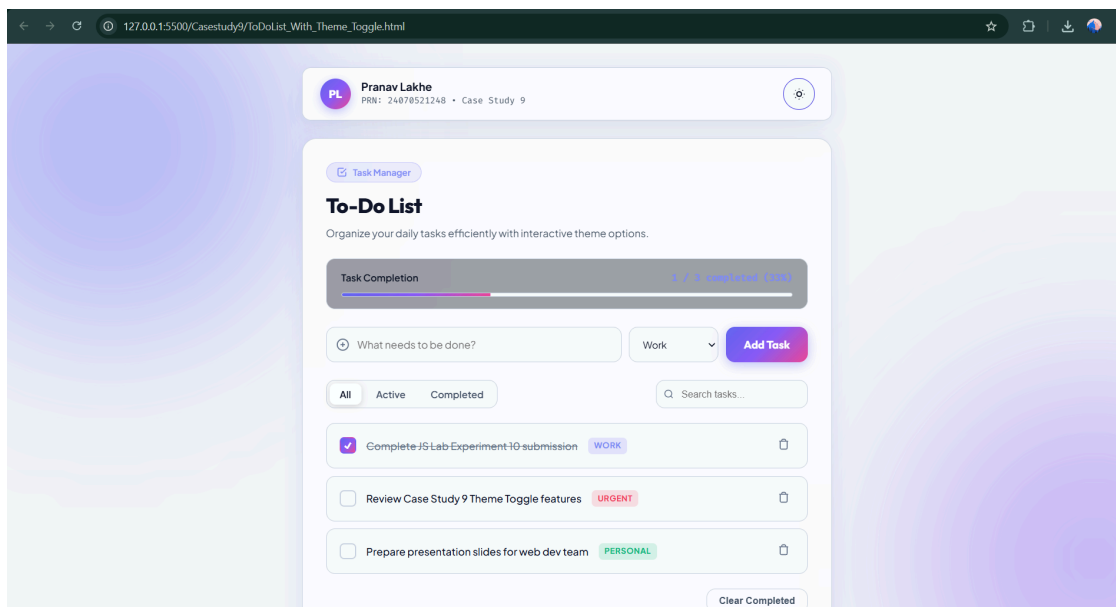
function updateMetrics() {
  const total = tasks.length;
  const completed = tasks.filter(t => t.completed).length;
  const percentage = total === 0 ? 0 : Math.round((completed / total) * 100);
  statsText.textContent = `${completed} / ${total} completed (${percentage}%)`;
  progressBar.style.width = `${percentage}%`;
}
// Full source: CaseStudy/Case Study 9 (24070521248 Pranav Lakhe)/ToDoList_With_Theme_Toggle.html

```

Output

Tasks display with a colored category badge and a checkbox to mark completion (strikethrough style applied). The search box filters the visible list by text match in real time, and the All/Active/Completed buttons narrow it further. A progress bar and "X / Y completed" counter update live. On reload, previously added tasks are restored from localStorage.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Casestudy9/ToDoList_With_Theme_Toggle.html

Result/Conclusion

This case study successfully demonstrates persistent client-side state using `localStorage` (surviving page reloads without a backend), combined search-and-filter logic over an array, and category-based UI badging — extending the basic to-do list pattern into a more complete task-management tool.

Experiment 10

Experiment Title

Student Information Portal — Fetching JSON with fetch() and jQuery \$.getJSON()

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Fetch API)
- jQuery 3.7.1 (CDN)

Experiment Program Code

```
// Method A: Using Vanilla JavaScript fetch() API
function loadUsingFetch() {
    fetch('students.json')
        .then(response => {
            if (!response.ok) throw new Error(`HTTP error! status: ${response.status}`);
            return response.json();
        })
        .then(data => { renderTable(data); updateMetrics(data); })
        .catch(error => console.error('Fetch error:', error));
}

// Method B: Using jQuery $.getJSON() API
function loadUsingjQuery() {
    $.getJSON('students.json', function(data) {
        renderTable(data);
        updateMetrics(data);
    }).fail(function(jqxhr, textStatus, error) {
        console.error('jQuery $.getJSON error:', textStatus, error);
    });
}

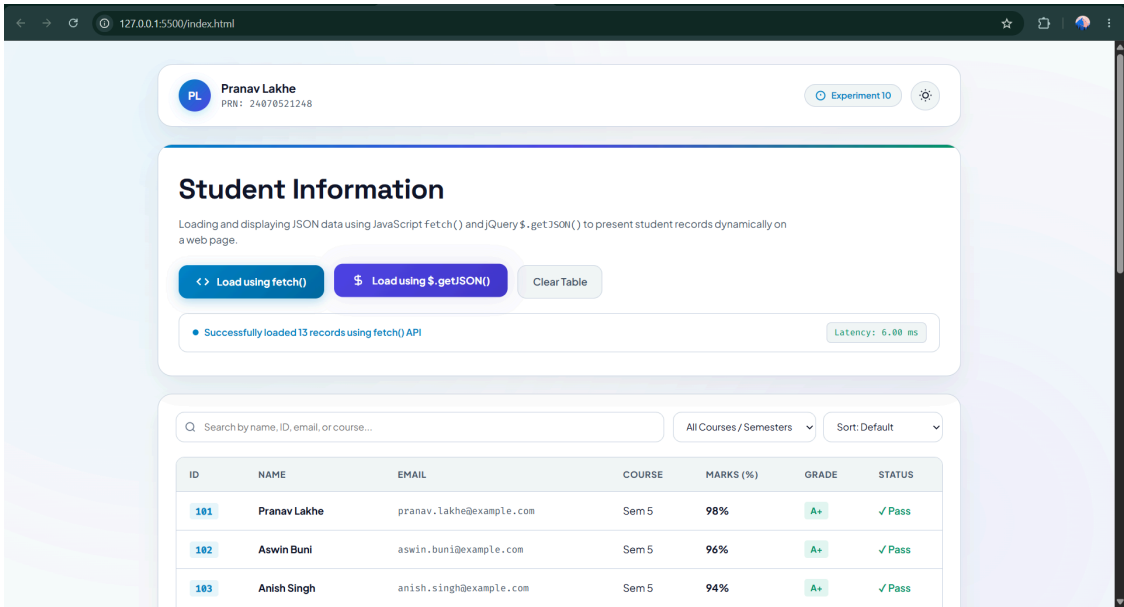
function renderTable(data) {
    let rowsHTML = '';
    data.forEach(student => {
        let gradeClass = 'grade-a';
        if (student.marks < 75 && student.marks >= 60) gradeClass = 'grade-b';
        if (student.marks < 60) gradeClass = 'grade-c';
        rowsHTML += `<tr>
            <td>${student.id}</td><td>${student.name}</td>
            <td><a href="mailto:${student.email}">${student.email}</a></td>
            <td>${student.course}</td><td>${student.marks}%</td>
            <td><span class="badge-grade ${gradeClass}">${student.grade}</span></td>
            <td>${student.status}</td></tr>`;
    });
    studentTableBody.innerHTML = rowsHTML;
}

// Latency-timed fetch button handler
btnFetchVanilla.addEventListener('click', () => {
    const t0 = performance.now();
    fetch('students.json')
        .then(response => { if (!response.ok) throw new Error('HTTP error ' + response.status); return response.json(); })
        .then(data => {
            const latency = (performance.now() - t0).toFixed(2);
            rawStudentsData = data;
            applyFilters();
            statusText.textContent = `Successfully loaded ${data.length} records using fetch() API`;
            latencyBadge.textContent = `Latency: ${latency} ms`;
        });
});
```

Output

Clicking the fetch() or jQuery \$.getJSON() button loads students.json and renders a searchable, sortable, course-filterable table of student records (ID, name, email, course, marks, grade badge, status), along with a latency badge showing how long the request took. Both methods produce identical rendered output, illustrating the two approaches to asynchronous JSON retrieval.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: index.html & students.json

Result/Conclusion

The experiment successfully demonstrates two equivalent approaches to fetching external JSON data — the native Promise-based fetch() API and jQuery's \$.getJSON() — including error handling via .catch()/fail() and dynamic table rendering, confirming both methods integrate cleanly with client-side filtering and sorting.

Case Study 10

Case Study Title

Campus Event Registration System — Dual-Method JSON Fetching with Search, Filter & Diagnostics

Aim

To build a client-side event-registration dashboard that loads student registration records from a JSON source using either the native fetch() API or jQuery's \$.getJSON(), letting the user compare both methods while searching, filtering by status, and viewing live request diagnostics.

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Fetch API)
- jQuery 3.7.1 (CDN)

Project Description

The dashboard lets the user pick a load method (fetch or jQuery) via radio buttons, then load real data or deliberately trigger an error (missing_students_data.json) to observe failure handling. A diagnostics panel reports the last request's status, source URL, method used, and latency in milliseconds. Loaded records populate a searchable, status-filterable table with summary stat cards (Total, Registered, Pending, Cancelled), and a persistent light/dark theme toggle is included.

Case Study Program Code

```
// 1. Native JS fetch()
function fetchStudentDataNative(url, startTime) {
  fetch(url)
    .then(response => {
      if (!response.ok) throw new Error(`HTTP Error ${response.status}: ${response.statusText} ||
'File Not Found'`);
      return response.json();
    })
    .then(data => handleDataLoadSuccess(data, 'JavaScript fetch()', Math.round(performance.now() -
startTime), url))
    .catch(error => handleDataLoadError(error.message, 'JavaScript fetch()',
Math.round(performance.now() - startTime), url));
}

// 2. jQuery $.getJSON()
function fetchStudentDatajQuery(url, startTime) {
  $.getJSON(url)
    .done(data => handleDataLoadSuccess(data, 'jQuery $.getJSON()', Math.round(performance.now() -
startTime), url))
    .fail((jqxhr, textStatus, errorThrown) => {
      const status = jqxhr.status ? `HTTP ${jqxhr.status}` : 'Network / Parse Error';
      handleDataLoadError(`Failed to load "${url}": ${status}`, 'jQuery $.getJSON()',
Math.round(performance.now() - startTime), url);
    });
}

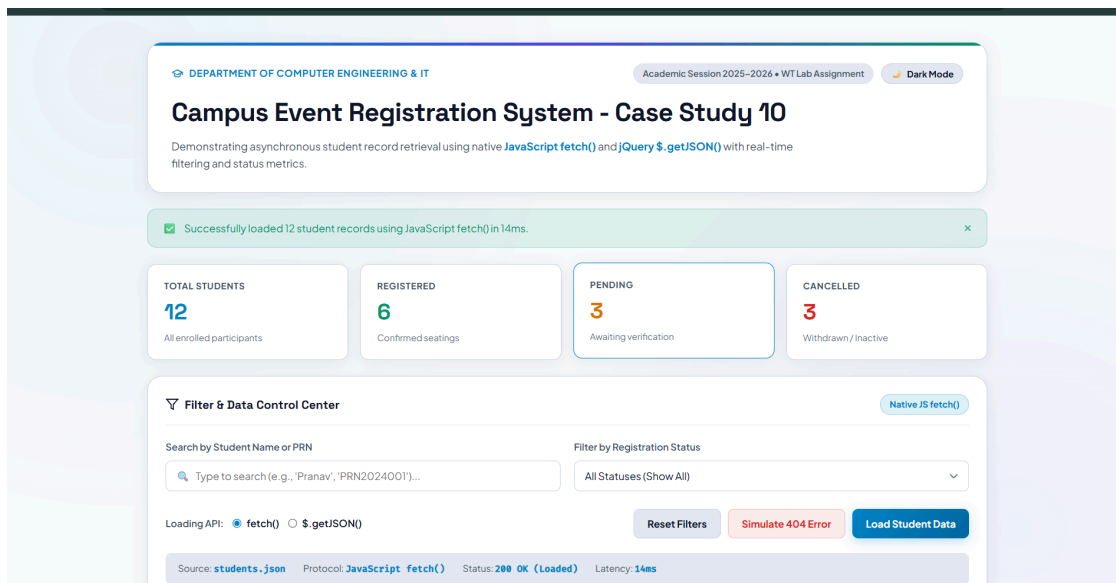
function applyFilters() {
  const searchTerm = document.getElementById('searchInput')?.value.trim().toLowerCase() || '';
  const selectedStatus = document.getElementById('statusFilter')?.value || 'All';
  const filtered = allStudents.filter(student => {
    const matchesSearch = searchTerm === '' || (student.studentName ||
'').toLowerCase().includes(searchTerm) || (student.prn || '').toLowerCase().includes(searchTerm);
    const matchesStatus = selectedStatus === 'All' || student.registrationStatus ===
selectedStatus;
    return matchesSearch && matchesStatus;
  });
  renderTable(filtered);
}

function toggleTheme() {
  const nextTheme = currentTheme === 'light' ? 'dark' : 'light';
  setTheme(nextTheme);
}
// Full source: CaseStudy/Case Study10 (24070521248 Pranav Lakhe)/index.html, script.js, students.json
```

Output

Loading valid data populates the table and stat cards, and shows a green success banner with record count, method, and latency. Triggering the error case shows a red error banner and empty-state message. The search box and status dropdown filter the table live, and "Clear Filters" resets both.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy10/index.html

Result/Conclusion

This case study successfully demonstrates both major approaches to asynchronous JSON retrieval (fetch vs. jQuery), explicit success/error branching with user-facing diagnostics, and combined search-and-filter logic over fetched data — providing a realistic, comparative view of client-side data loading strategies.